

PrismViz 组件交付说明

PrismViz 是面向甲方的单组件交付包。当前交付目标不是完整联动系统，而是提供五个可独立调用的可视化组件、两套示例数据、原生 HTML demo、React demo，以及一份从数据到调用的完整说明。

交付内容

PrismViz/	
src/	组件源码、数据建模、数据适配、详情格式化、运行时依赖
instances/	
reference/	reference 数据、原生 demo、React demo
client/	client 数据、原生 demo、React demo
shared/	reference/client demo 共用页面外壳样式
example/	原系统交互示例，仅用于参考 legacy 效果
README.md	唯一主文档
README.pdf	README 的 PDF 版本

包含的五个组件：

- Tree：slice/topic 层级列表入口。由于产业通已经包含了较好的层次树可视化，故本组件仅做了层次展开，以做可视效果
- Chord：slice 之间的关系总览。
- Prism：三维棱镜式多 slice 总览。
- Scroll：单个 slice 内部点边图，支持 time layered 或 free graph。
- List：节点、边、slice、stream 等点击结果的详情面板。

边界说明：

- src/ 只保留组件自身功能、数据建模、数据适配和详情格式化。
- instances/ 只保留各组件自己的 demo 和对应数据的可视化实例。
- instances/shared/ 只服务 demo 页面外壳，不属于组件 API。
- src/runtime/ 保存 Prism/Scroll legacy 渲染需要的轻量运行时依赖。
- example/ 保留源系统示例，用来查看原系统交互效果，不作为新组件依赖。
- 不包含 integrated demo、combinations demo、组件间 coordinator、旧合包数据 prismviz_data.json。
- 单组件 demo 不依赖 example/prismviz.js。

两套数据

reference 数据

instances/reference/data/ 是原 GeneticPrism/reference 学术数据的组件化版本。它以学术论文为节点、引用关系为边，slice 表示论文主题/研究方向。该数据用于复现原系统中 Prism、Scroll、Chord、List、Tree 的视觉效果，尤其是 Scroll 的 time layered 布局。

reference 数据文件：

```
instances/reference/data/  
  slice_definitions.json  
  slice_hierarchy.json  
  entities.json  
  relations.json  
  display_info.json  
  component_options.json
```

client 数据

`instances/client/data/` 是甲方业务形态的示例数据。它以企业/对象为节点，以投资、股权或其他业务关系为边，slice 表示产业环节或业务分类。client 数据用于演示甲方数据接入组件后的效果。

client 数据文件和 reference 保持一致：

```
instances/client/data/  
  slice_definitions.json  
  slice_hierarchy.json  
  entities.json  
  relations.json  
  display_info.json  
  component_options.json
```

两套数据的目标是进入统一模型后走同一批组件入口。差异只发生在最前面的 adapter：reference 主要保留原学术语义；client 会把产业/企业字段整理成组件统一字段，并对同一点之间的多条关系做聚合保留。

快速打开 Demo

不要直接双击 HTML。请在仓库根目录启动静态服务：

```
cd PrismViz  
python3 -m http.server 8765
```

reference 原生 demo：

```
http://127.0.0.1:8765/instances/reference/demos/tree.html  
http://127.0.0.1:8765/instances/reference/demos/chord.html  
http://127.0.0.1:8765/instances/reference/demos/prism.html  
http://127.0.0.1:8765/instances/reference/demos/scroll.html  
http://127.0.0.1:8765/instances/reference/demos/list.html
```

client 原生 demo：

```
http://127.0.0.1:8765/instances/client/demos/tree.html  
http://127.0.0.1:8765/instances/client/demos/chord.html  
http://127.0.0.1:8765/instances/client/demos/prism.html  
http://127.0.0.1:8765/instances/client/demos/scroll.html  
http://127.0.0.1:8765/instances/client/demos/list.html
```

reference React demo：

```
http://127.0.0.1:8765/instances/reference/demos-react/tree.html  
http://127.0.0.1:8765/instances/reference/demos-react/chord.html  
http://127.0.0.1:8765/instances/reference/demos-react/prism.html  
http://127.0.0.1:8765/instances/reference/demos-react/scroll.html  
http://127.0.0.1:8765/instances/reference/demos-react/list.html
```

client React demo：

```
http://127.0.0.1:8765/instances/client/demos-react/tree.html  
http://127.0.0.1:8765/instances/client/demos-react/chord.html  
http://127.0.0.1:8765/instances/client/demos-react/prism.html  
http://127.0.0.1:8765/instances/client/demos-react/scroll.html  
http://127.0.0.1:8765/instances/client/demos-react/list.html
```

缓存与发布

联调阶段建议服务器对 `html/js/css/json` 返回 `Cache-Control: no-cache`，避免浏览器继续使用旧 `module` 或旧数据。本地调试时可以打开浏览器 DevTools，在 Network 面板勾选 `Disable cache`；如果改过文件名、删除文件或调整 `import`，建议使用硬刷新。

正式交付建议使用版本目录或版本 tag，例如 `PrismViz-v1.0.0/`。版本变化后浏览器会按新路径加载资源，避免新旧代码混用。

数据到调用的完整链路

推荐把甲方数据整理成 6 个对象或 6 个 JSON 文件：

```
slice_definitions.json  -> slices
slice_hierarchy.json   -> hierarchy
entities.json          -> entities
relations.json         -> relations
display_info.json      -> displayInfo
component_options.json -> config
```

调用链路：

```
原始 JSON
-> src/adapters/reference.js 或 src/adapters/client.js
-> src/core/model.js 的统一建模
-> model.tree / model.chord / model.prism / model.scrollBySliceId / model.list
-> renderTree / renderChord / renderPrism / renderScroll / renderList
-> 或 ReactTree / ReactChord / ReactPrism / ReactScroll / ReactList
```

正式接入时，甲方不需要复制或编写 `demo-support`。当前 demo 页面只是自己 `fetch` 本地 JSON，然后把数据对象传给 `src/adapters` 或 `src/core`。

数据格式

最小输入对象：

```
const input = {
  slices,
  hierarchy,
  entities,
  relations,
  displayInfo,
  config,
};
```

最小 slice：

```
{
  "id": "slice_25351",
  "name": "Fabless设计",
  "shortName": "Fabless设计",
  "color": "#f6b35a"
}
```

最小节点：

```
{
  "id": "149048320",
  "label": "北京电科智芯科技有限公司",
  "primarySliceId": "slice_25351",
  "sliceWeights": {"slice_25351": 1},
  "impact": 5000,
  "importance": 0.0001,
  "time": "2020",
  "meta": {"法人": "张鹏", "状态": "存续"}
}
```

最小关系:

```
{
  "id": "rel_001",
  "source": "149048320",
  "target": "214995632",
  "type": "investment",
  "weight": 1,
  "time": "2020",
  "meta": {"持股比例": "100.0%"}
}
```

`display_info.json` 用于控制 List Panel 展示字段, 建议预处理后只放语义最相关的内容:

```
{
  "entityInfoById": {
    "149048320": {
      "title": "北京电科智芯科技有限公司",
      "subtitle": "110100",
      "metrics": {"注册资本(万元)": 5000},
      "attributes": {"法人": "张鹏", "状态": "存续"}
    }
  },
  "relationInfoById": {
    "rel_001": {
      "title": "投资关系",
      "subtitle": "2020",
      "attributes": {"持股比例": "100.0%"}
    }
  }
}
```

Adapter 入口

推荐统一从 `src/adapters/index.js` 引入:

```
import {
  normalizeReferenceInput,
  buildReferenceModels,
  referenceDetailRows,
  getReferenceDisplayInfo,
  normalizeClientInput,
  buildClientModels,
  capScrollModelBySliceWeights,
  clientDetailRows,
  getClientDisplayInfo,
} from "./src/adapters/index.js";
```

也可以按数据类型直接引入：

```
import {
  buildReferenceModels,
  detailRows as referenceDetailRows,
} from "../src/adapters/reference.js";
import {
  buildClientModels,
  detailRows as clientDetailRows,
} from "../src/adapters/client.js";
```

reference:

```
const model = buildReferenceModels({
  slices,
  hierarchy,
  entities,
  relations,
  displayInfo,
  config,
});
```

client:

```
const model = buildClientModels({
  slices,
  hierarchy,
  entities,
  relations,
  displayInfo,
  config,
});
```

高级底层入口一般不需要使用。常规接入建议始终走 `reference.js` 或 `client.js`，只有在已经自行完成标准化数据建模时，才直接使用 `core`：

```
import {buildComponentModels} from "../src/core/index.js";

const model = buildComponentModels({
  normalized: true,
  slices,
  hierarchy,
  entities,
  relations,
  displayInfo,
  config,
});
```

原生 HTML 调用

原生调用最小模板：

```
<link rel="stylesheet" href="../src/components/tree/tree.css">
<div id="tree" style="width: 960px; height: 640px;"></div>
<script type="module" src="../demo.js"></script>
```

```
import {buildClientModels} from "./src/adapters/index.js";
import {renderTree} from "./src/components/tree/Tree.js";

const model = buildClientModels(input);

renderTree("#tree", model.tree, {
  maxLevel: 2,
  showCount: true,
  onSelect: node => console.log(node),
});
```

Scroll、Prism、Chord 这类可视化组件的外层容器必须有明确宽高。 `instances/*/demos/*.html` 是可直接参考的完整原生调用示例。

`instances/shared/demo.css` 和 `instances/shared/react-demo.css` 只用于交付包 demo 页面外壳。甲方正式集成时应优先引用 `src/components/*.css` 和对应组件入口。

React 调用

React 工程里引入 wrapper:

```
import {ReactTree} from "./src/components/tree/Tree.react.js";
import {ReactChord} from "./src/components/chord/Chord.react.js";
import {ReactPrism, ReactPrismSliceTags} from "./src/components/prism/Prism.react.js";
import {ReactScroll} from "./src/components/scroll/Scroll.react.js";
import {ReactList} from "./src/components/list/List.react.js";
```

同时引入样式:

```
import "./src/components/tree/tree.css";
import "./src/components/chord/chord.css";
import "./src/components/prism/prism.css";
import "./src/components/scroll/scroll.css";
import "./src/components/list/list.css";
```

React Scroll 最小示例:

```
function ScrollDemo({model, sliceId, onSelect}) {
  return (
    <div style={{height: 640}}>
      <ReactScroll
        data={model.scrollBySliceId[sliceId]}
        options={{
          layoutMode: "free-graph",
          scrollRenderer: "graph",
          withContext: true,
          showStreams: true,
          componentModels: model,
          slices: model.slices,
          colorMap: model.colorMap,
        }}
        onSelect={onSelect}
      />
    </div>
  );
}
```

`instances/*/demos-react/*.html` 是不依赖构建工具的 React CDN 示例。甲方正式工程可以把这些写法迁移到自己的 React/Vite/webpack 项目中。

组件 API

Tree

调用入口：

```
const treeInstance = renderTree(container, model.tree, options);
```

React：

```
<ReactTree data={model.tree} options={options} onSelect={handleSelect} />
```

常用 options：

- `maxLevel`：最多显示层级。
- `showCount`：是否显示数量。
- `onSelect`：选择节点回调。
- `onHover`：hover 节点回调。

实例方法：

- `update(nextData, nextOptions)`
- `destroy()`

说明：产业通系统已有较成熟的 Topic Tree/产业层级树实现；原系统中也没有深度开发 Tree 组件。因此本包里的 Tree 是为了配合组件展示和后续联动做的轻量列表实现。

Chord

调用入口：

```
const chordInstance = renderChord(container, model.chord, options);
```

React：

```
<ReactChord data={model.chord} options={options} onSelect={handleSelect} />
```

常用 options：

- `onSelect`：点击 slice 或 ribbon。
- `onSliceHover`：hover slice。
- `onRelationHover`：hover slice 间关系。

实例方法：

- `highlightSlice(sliceId)`
- `highlightRelation(sourceSliceId, targetSliceId)`
- `clearHighlight()`
- `update(nextData, nextOptions)`
- `destroy()`

Prism

调用入口：

```
const prismInstance = renderPrism(container, model.prism, options);  
const tagInstance = renderPrismSliceTags(tagContainer, model.slices, tagOptions);
```

React：

```
<ReactPrism data={model.prism} options={options} />
<ReactPrismSliceTags slices={model.slices} options={tagOptions} onSelect={handleSlice} />
```

常用 options:

- `onSelect`: 点击 Prism 元素。
- `activeSliceId`: 当前 slice。
- `colorMap`: slice 颜色。
- `widthMode`: `weighted` 或 `equal`。
- `faceGraphLayoutMode`: 面内布局方式。
- `autoRotate` / `rotationSpeed`: 自动旋转控制。

实例方法:

- `update(nextData, nextOptions)`
- `destroy()`

Scroll

调用入口:

```
const scrollInstance = renderScroll(container, model.scrollBySliceId[sliceId], options);
```

React:

```
<ReactScroll data={model.scrollBySliceId[sliceId]} options={options} onSelect={handleSelect} />
```

reference 支持:

- `layoutMode`: `"layered-time"`: time layered。
- `layoutMode`: `"free-graph"`: free graph。

client 当前只使用 `free-graph`。reference/client 的 free graph 使用同一套 Scroll 入口。

常用 options:

- `layoutMode`: `layered-time` 或 `free-graph`。
- `scrollRenderer`: `layered-scroll` 或 `graph`。
- `withContext`: 是否保留上下文关系。
- `showStreams`: 是否展示左右 stream。
- `componentModels`: 完整模型, 用于详情和跨 slice 查询。
- `slices`: slice 列表。
- `colorMap`: slice 颜色。

回调:

- `onSelect(item, context)`
- `onNodeHover(node, context)`
- `onEdgeHover(edge, context)`
- `onSegmentSelect(segment, context)`
- `onStreamSelect(stream, context)`

实例方法:

- `highlightEntity(entityId)`
- `highlightRelation(relationOrId)`
- `highlightContextSlice(sliceId, direction)`
- `update(nextData, nextOptions)`
- `destroy()`

Scroll 下方的 slice 标签不是 Scroll 内部 DOM，而是独立的 Prism 标签组件：

```
renderPrismSliceTags("#slice-tags", model.slices, {
  activeSliceId,
  colorMap: model.colorMap,
  onSelect: slice => {},
});
```

client Scroll 如果需要限制单个 slice 的节点数，可以使用：

```
const scrollModel = capScrollModelBySliceWeights(model, sliceId, 300);
```

List

调用入口：

```
const listInstance = renderList(container, detailData, options);
```

React：

```
<ReactList data={detailData} options={options} />
```

常用 options：

- mode：json、list、table、auto。
- numberPrecision：数值精度。
- title：标题。

用法：

```
import {detailRows} from "../src/adapters/client.js";
// reference 数据对应从 "../src/adapters/reference.js" 引入同名 detailRows

renderList("#detail", detailRows(model, item, context), {
  mode: "json",
  numberPrecision: 4,
});
```

List 可以直接接收 Tree、Chord、Prism、Scroll 点击回调中的数据，也可以接收 detailRows(model, item, context) 的语义整理结果。对于 client 中同一点的多条关系，聚合边详情会显示 title: "多重关系"，并在 relations 字段下逐条展开原始 relation。List 是详情展示接收端，不进入组件间 coordinator 的联动规划。

组件间交互规划

本周计划实现独立 interaction coordinator。原则是组件之间不互相直接 import，由 coordinator 根据已挂载组件启用对应联动，从而做到“调用哪几个组件，就实现哪几个组件之间的交互”。

建议事件：

```
slice: hover
slice: select
slice: clear
entity: hover
entity: select
entity: clear
relation: hover
relation: select
relation: clear
contextSlice: hover
contextSlice: select
contextSlice: clear
view: change
layout: change
```

主要两两交互：

- Tree -> Prism：选择 slice 后 Prism 高亮/旋转到对应 face。
- Prism -> Tree：Prism 当前 face 变化时 Tree 同步 active 行。
- Tree -> Chord：选择 slice 后 Chord 高亮对应 arc/ribbon。
- Chord -> Tree：点击 Chord arc 后 Tree 高亮对应 slice。
- Tree -> Scroll：选择 slice 后 Scroll 切换到该 slice。
- Scroll -> Tree：Scroll 当前 slice 变化后 Tree 同步 active 行。
- Chord -> Prism：点击 Chord arc 后 Prism 高亮对应 face。
- Prism -> Chord：Prism 当前 slice 变化后 Chord 同步高亮。
- Chord -> Scroll：点击 Chord arc/ribbon 后 Scroll 切换 slice 或高亮 context。
- Scroll -> Chord：Scroll stream hover 时 Chord 临时高亮相关 slice relation。
- Prism -> Scroll：点击 Prism face 或 slice tag 后进入对应 Scroll。
- Scroll -> Prism：从 Scroll 返回或切换 slice 后 Prism 同步当前 slice。

原系统可参考 `example/index.html` 和 `example/prismviz.js`。其中典型链路包括：

- `bindTreePanel()`：左侧 topic/slice 列表点击。
- `handleSliceTagClick()`：点击 slice tag/tree/prism 后切换当前 slice 或 Scroll 视图。
- `updateOpacity()`：Prism 旋转时判断当前可见面并同步 active slice。
- `syncChordSelectionBySliceId()`：Chord 根据当前 slice 高亮 arc/ribbon。
- `mouseoverStreamSegment()`：Scroll 左右 stream hover 时高亮上下文边。

接入建议

最稳妥的联调方式：

1. 将整个 `PrismViz/` 放进甲方 React 工程根目录。
2. 先用 `instances/client/data/` 的格式替换为甲方真实数据。
3. 调用 `buildClientModels()` 生成统一模型。
4. 先接入单个组件，例如 Scroll 或 Prism。
5. 确认单组件数据格式和视觉效果稳定后，再按组件间交互规划实现 Tree/Chord/Prism/Scroll 的 coordinator。
6. 后续如果需要做成内部 npm 包，可以在接入稳定后再封装。

验收检查

当前交付包应满足：

- `src/adapters/reference.js` 和 `src/adapters/client.js` 对称命名。
- `reference/client` demo 共用 `instances/shared/` 下的 demo 页面样式。
- Prism/Scroll demo 从 `src/runtime/` 读取运行时依赖，不从 `example/` 读取。
- `instances/` 下没有 `demo-support` 目录。

- demo 页面直接读取本地 JSON，不要求甲方复制 demo-support。
- docs/ 不再恢复，主说明集中在 README.md 和 README.pdf 。
- example/ 保留为源系统参考。
- src/components/scroll/legacyLayeredScroll.js 保留，因为 reference Scroll 的 time layered 仍依赖它。